

29

FLOW CONTROL: LOOPING WITH WHILE/UNTIL



In the previous chapter, we developed a menu-driven program to produce various kinds of system information. The program works, but it still has a significant usability problem. It executes only a single choice and then terminates. Even worse, if an invalid selection is made, the program terminates with an error, without giving the user an opportunity to try again. It would be better if we could somehow construct the program so that it could repeat the menu display and selection over and over, until the user chooses to exit the program.

In this chapter, we will look at a programming concept called *looping*, which can be used to make portions of programs repeat. The shell provides three compound commands for looping. We will look at two of them in this chapter and the third in a later chapter.

Looping

Daily life is full of repeated activities. Going to work each day, walking the dog, and slicing a carrot are all tasks that involve repeating a series of steps. Let's consider slicing a carrot. If we express this activity in pseudocode, it might look something like this:

1. Get cutting board.
2. Get knife.
3. Place carrot on cutting board.
4. Lift knife.
5. Advance carrot.
6. Slice carrot.
7. If entire carrot sliced, then quit; else go to step 4.

Steps 4 through 7 form a *loop*. The actions within the loop are repeated until the condition, "entire carrot sliced," is reached.

while

bash can express a similar idea. Let's say we wanted to display five numbers in sequential order from 1 to 5. A bash script could be constructed as follows:

```
#!/bin/bash

# while-count: display a series of numbers

count=1

while [[ "$count" -le 5 ]]; do
    echo "$count"
    count=$((count + 1))
done
echo "Finished."
```

When executed, this script displays the following:

```
[me@linuxbox ~]$ while-count
1
2
3
4
5
Finished.
```

The syntax of the `while` command is as follows:

```
while commands; do commands; done
```

Like `if`, `while` evaluates the exit status of a list of commands. As long as the exit status is zero, it performs the commands inside the loop. In the previous script, the variable `count` is created and assigned an initial value of 1. The `while` command evaluates the exit status of the `[[ ]]` compound command. As long as the `[[ ]]` command returns an exit status of zero, the commands within the loop are executed. At the end of each cycle, the `[[ ]]` command is repeated. After five iterations of the loop, the value of `count` has increased to 6, the `[[ ]]` command no longer returns an exit status of zero, and the loop terminates. The program continues with the next statement following the loop.

We can use a *while loop* to improve the `read-menu` program from the previous chapter.

```
#!/bin/bash

# while-menu: a menu driven system information program

DELAY=3 # Number of seconds to display results

while [[ "$REPLY" != 0 ]]; do
    clear
    cat << _EOF_
        Please Select:

            1. Display System Information
            2. Display Disk Space
            3. Display Home Space Utilization
            0. Quit

_EOF_
    read -r -p "Enter selection [0-3] > "

    if [[ "$REPLY" =~ ^[0-3]$ ]]; then
        if [[ $REPLY == 1 ]]; then
            echo "Hostname: $HOSTNAME"
            uptime
            sleep "$DELAY"
        fi
        if [[ "$REPLY" == 2 ]]; then
            df -h
            sleep "$DELAY"
        fi
        if [[ "$REPLY" == 3 ]]; then
            if [[ "$(id -u)" -eq 0 ]]; then
                echo "Home Space Utilization (All Users)"
                du -sh /home/*
            else
                echo "Home Space Utilization ($USER)"
                du -sh "$HOME"
            fi
            sleep "$DELAY"
        fi
    else
        echo "Invalid entry."
```

```

        sleep "$DELAY"
    fi
done
echo "Program terminated."

```

By enclosing the menu in a while loop, we are able to have the program repeat the menu display after each selection. The loop continues as long as `REPLY` is not equal to 0 and the menu is displayed again, giving the user the opportunity to make another selection. At the end of each action, a `sleep` command is executed so the program will pause for a few seconds to allow the results of the selection to be seen before the screen is cleared and the menu is redisplayed. Once `REPLY` is equal to 0, indicating the “quit” selection, the loop terminates and execution continues with the line following `done`.

break and continue

`bash` provides two builtin commands that can be used to control program flow inside loops. The `break` command immediately terminates a loop, and program control resumes with the next statement following the loop. The `continue` command causes the remainder of the loop to be skipped, and program control resumes with the next iteration of the loop. Here we see a version of the while-menu program incorporating both `break` and `continue`:

```

#!/bin/bash

# while-menu2: a menu driven system information program

DELAY=3 # Number of seconds to display results

while true; do
    clear
    cat << _EOF_
        Please Select:

            1. Display System Information
            2. Display Disk Space
            3. Display Home Space Utilization
            0. Quit

_EOF_
    read -p "Enter selection [0-3] > "

    if [[ "$REPLY" =~ ^[0-3]$ ]]; then
        if [[ "$REPLY" == 1 ]]; then
            echo "Hostname: $HOSTNAME"
            uptime
            sleep "$DELAY"
            continue
        fi
        if [[ "$REPLY" == 2 ]]; then
            df -h
            sleep "$DELAY"

```

```

        continue
    fi
    if [[ "$REPLY" == 3 ]]; then
        if [[ "$(id -u)" -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/*
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME"
        fi
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == 0 ]]; then
        break
    fi
else
    echo "Invalid entry."
    sleep "$DELAY"
fi
done
echo "Program terminated."

```

In this version of the script, we set up an *endless loop* (one that never terminates on its own) by using the `true` command to supply an exit status of zero to `while`. Since `true` will always exit with an exit status of zero, the loop will never end. This is a surprisingly common scripting technique. Since the loop will never end on its own, it's up to the programmer to provide some way to break out of the loop when the time is right. In this script, the `break` command is used to exit the loop when the 0 selection is chosen. The `continue` command has been included at the end of the other script choices to allow for more efficient execution. By using `continue`, the script will skip over code that is not needed when a selection is identified. For example, if the 1 selection is chosen and identified, there is no reason to test for the other selections.

select

This would be a good time to mention the `select` shell builtin, which is used to create looping menus. It has a syntax that looks like this, where *var* is a variable and *string* is the text of a menu choice:

```
select var in [string... ;] do commands; done
```

When `select` executes, it displays the *string*(s) followed by the contents of the `PS3` (prompt string 3) variable as a prompt for the user's input. Once a choice is made, it sets the `REPLY` variable with the user's input (just like with `read`) and returns the string associated with the choice in the variable *var*. Once the values are set, commands are performed, and the prompt is

displayed again for another choice. This sounds a little confusing, but we can demonstrate it with this tiny script:

```
#!/bin/bash

# select-demo: select builtin demo

PS3="
Your choice: "

select my_choice in First Second Third Fourth Quit; do
    echo "REPLY= $REPLY  my_choice= $my_choice"
    [[ "$my_choice" == "Quit" ]] && break
done
```

First, we set the contents of the PS3 variable with our desired prompt string. Next, we execute select. In this example, we have five strings and though we have used single words as our strings, we can use any kind of quoted text. For our *commands*, we simply echo the assignments made by select. We also test the contents of our variable my_choice to see if the user has chosen the Quit option, and if so, we perform a break to exit the loop.

```
[me@linuxbox ~]$ select-demo
1) First
2) Second
3) Third
4) Fourth
5) Quit
```

Your choice:

When select first executes, it displays each of our strings preceded by a number followed by our prompt string. The user next enters the number representing the desired choice. The select command then sets the REPLY variable to contain whatever the user entered and the corresponding string if any.

```
Your choice: 1
REPLY= 1  my_choice= First
```

```
Your choice: 2
REPLY= 2  my_choice= Second
```

Here we see the user entered 1, and the echo command displays the values of the REPLY and my_choice variables. select will repeat displaying the prompt string until the user enters a 5. If the user enters an invalid value, select sets my_choice to an empty string. If the user simply presses ENTER, this will cause select to start over and redisplay the list of menu choices.

```
Your choice: 6
REPLY= 6  my_choice=
```

```
Your choice: abc
REPLY= abc  my_choice=
```

The select loop will continue indefinitely until either a break command is encountered or the user types CTRL-D to signal end-of-file.

```
Your choice: 5
REPLY= 5 my_choice= Quit
[me@linuxbox ~]$
```

One interesting feature of select is that it does not display its menu choices or prompt string on standard output; rather, it uses standard error. This is actually handy because it allows the real work done by the commands within the loop to be redirected. Here's an example:

```
[me@linuxbox ~]$ select-demo > choices.txt
```

When we do this redirection, the menu and prompt are still displayed, but the output of the echo command is redirected.

Let's make an alternate version of our system information script replacing our previous while loop with select.

```
#!/bin/bash

# select-menu: a menu driven system information program

DELAY=3 # Number of seconds to display results
PS3="
Enter selection [1-4] > "

select str in \
    "Display System Information" \
    "Display Disk Space" \
    "Display Home Space Utilization" \
    "Quit"; do
    if [[ "$REPLY" == "1" ]]; then
        echo "Hostname: $HOSTNAME"
        uptime
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == "2" ]]; then
        df -h
        sleep "$DELAY"
        continue
    fi
    if [[ "$REPLY" == "3" ]]; then
        if [[ $(id -u) -eq 0 ]]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/* 2> /dev/null
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME" 2> /dev/null
        fi
        sleep "$DELAY"
        continue
    fi
done
```

```
fi
if [[ "$REPLY" == "4" ]]; then
    break
fi
if [[ -z "$str" ]]; then
    echo "Invalid entry."
    sleep "$DELAY"
fi
done
echo "Program terminated."
```

In our alternate script, we set the `PS3` variable and then invoke `select` with four strings. Though we could subsequently test the `str` variable set by `select`, it's easier to test the `REPLY` variable and act accordingly. At the end of the loop we check if the `str` variable has a zero length indicating an invalid value.

So, which method should we use for constructing a menu? The `select` command is interesting, but besides its use of standard error for the menu display, it doesn't really save us much, if any, coding effort, and it sharply limits the visual design of the menu display.

until

The `until` command is much like `while`, except instead of exiting a loop when a non-zero exit status is encountered, it does the opposite. An *until loop* continues until it receives a zero exit status. In our `while-count` script, we continued the loop as long as the value of the `count` variable was less than or equal to 5. We could get the same result by coding the script with `until`.

```
#!/bin/bash

# until-count: display a series of numbers

count=1

until [[ "$count" -gt 5 ]]; do
    echo "$count"
    count=$((count + 1))
done
echo "Finished."
```

By changing the test expression to `$count -gt 5`, `until` will terminate the loop at the correct time. The decision of whether to use the `while` or `until` loop is usually a matter of choosing the one that allows the clearest test to be written.

Reading Files with Loops

`while` and `until` can process standard input. This allows files to be processed with `while` and `until` loops. In the following example, we will display the contents of the *distros.txt* file used in earlier chapters:

```
#!/bin/bash

# while-read: read lines from a file

while read -r distro version release; do
    printf "Distro: %s\tVersion: %s\tReleased: %s\n" \
        "$distro" \
        "$version" \
        "$release"
done < distros.txt
```

To redirect a file to the loop, we place the redirection operator after the `done` statement. The loop will use `read` to input the fields from the redirected file. The `read` command will exit after each line is read, with a zero exit status until end-of-file is reached. At that point, it will exit with a non-zero exit status, thereby terminating the loop. It is also possible to pipe standard input into a loop.

```
#!/bin/bash

# while-read2: read lines from a file

sort -k 1,1 -k 2n distros.txt | while read -r distro version release; do
    printf "Distro: %s\tVersion: %s\tReleased: %s\n" \
        "$distro" \
        "$version" \
        "$release"
done
```

Here we take the output of the `sort` command and display the stream of text. However, it is important to remember that since a pipe will execute the loop in a subshell, any variables created or assigned within the loop will be lost when the loop terminates.

Summing Up

With the introduction of loops and our previous encounters with branching, subroutines, and sequences, we have covered the major types of flow control used in programs. `bash` has some more tricks up its sleeve, but they are refinements on these basic concepts.